



Sistema de Gestión de Torneos de Tiro con Arco

De la idea a la realidad: una solución integral desarrollada con SQL,
Backend y Frontend.

Grupo 1

Integrantes:

- Martín Alvayay
- Martin Araya
- José Ceardi
- Benjamín Letelier
- Benjamín Paredes
- Nicolas Rojas

El Desafío: Organizar Torneos de Tiro con Arco

Gestionar torneos de tiro con arco implica coordinar inscripciones, puntuaciones, clasificaciones y comunicaciones de forma simultánea. Sin una herramienta adecuada, los errores manuales y la desorganización afectan tanto a los organizadores como a los competidores.

Para los Organizadores

Reducir la carga administrativa y minimizar errores en la gestión del evento.

Para los Arqueros

Acceso ágil a inscripciones, resultados y clasificaciones en tiempo real.

https://www.worldarchery.sport/rhttps://www.worldarchery.sport/rulebook/article/793?book_title=Cap%C3%ADtulo%2011%20-%20Equipo%20de%20los%20Atletas

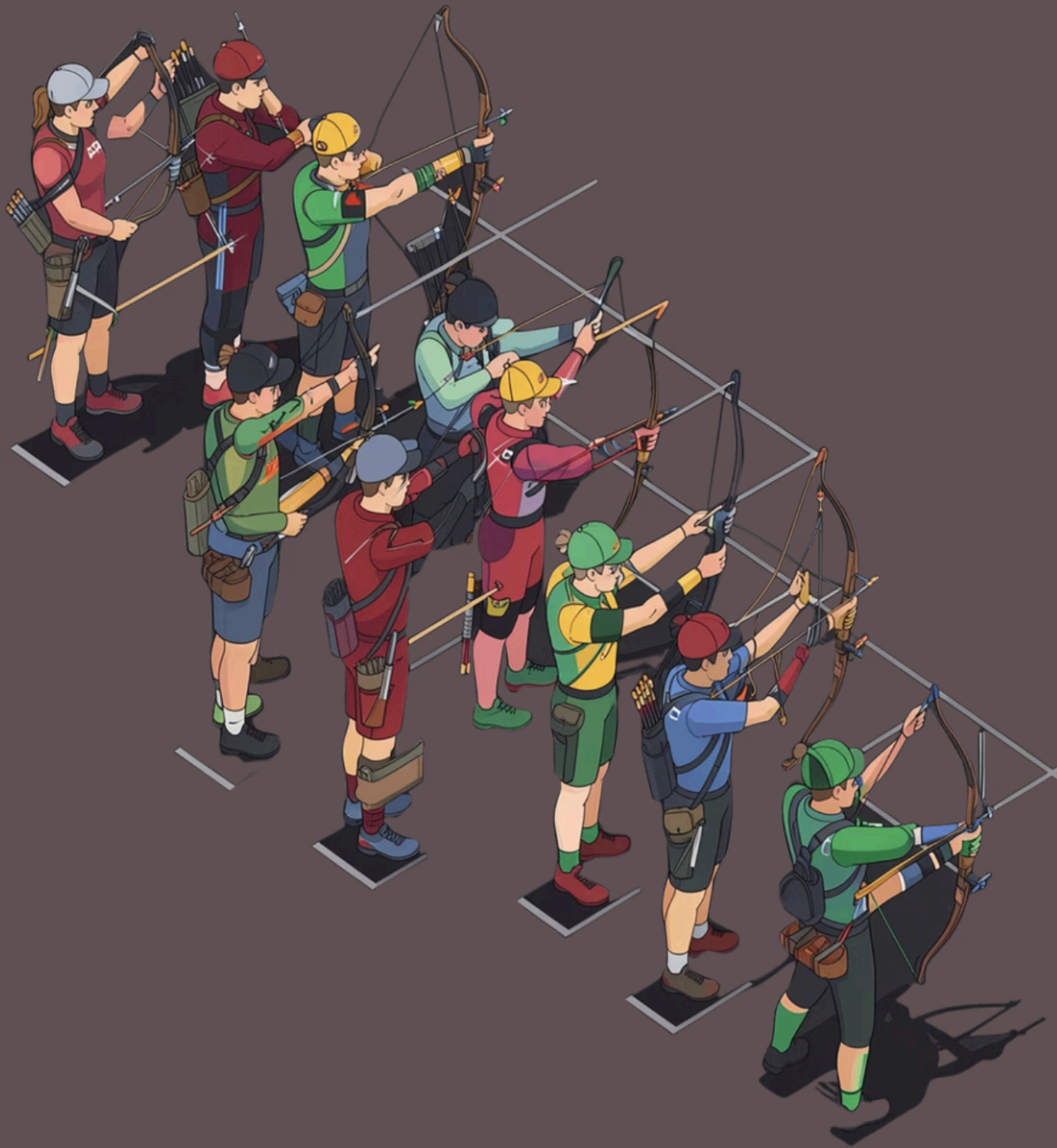


Diagrama de la arquitectura

Arquitectura por capas

Diseñamos un sistema integral que cubre todo el ciclo del torneo, estructurado sobre tres pilares tecnológicos fundamentales



Base de Datos SQL

Almacenamiento seguro y estructurado de toda la información.



Backend

Lógica de negocio, APIs REST y procesamiento de datos.



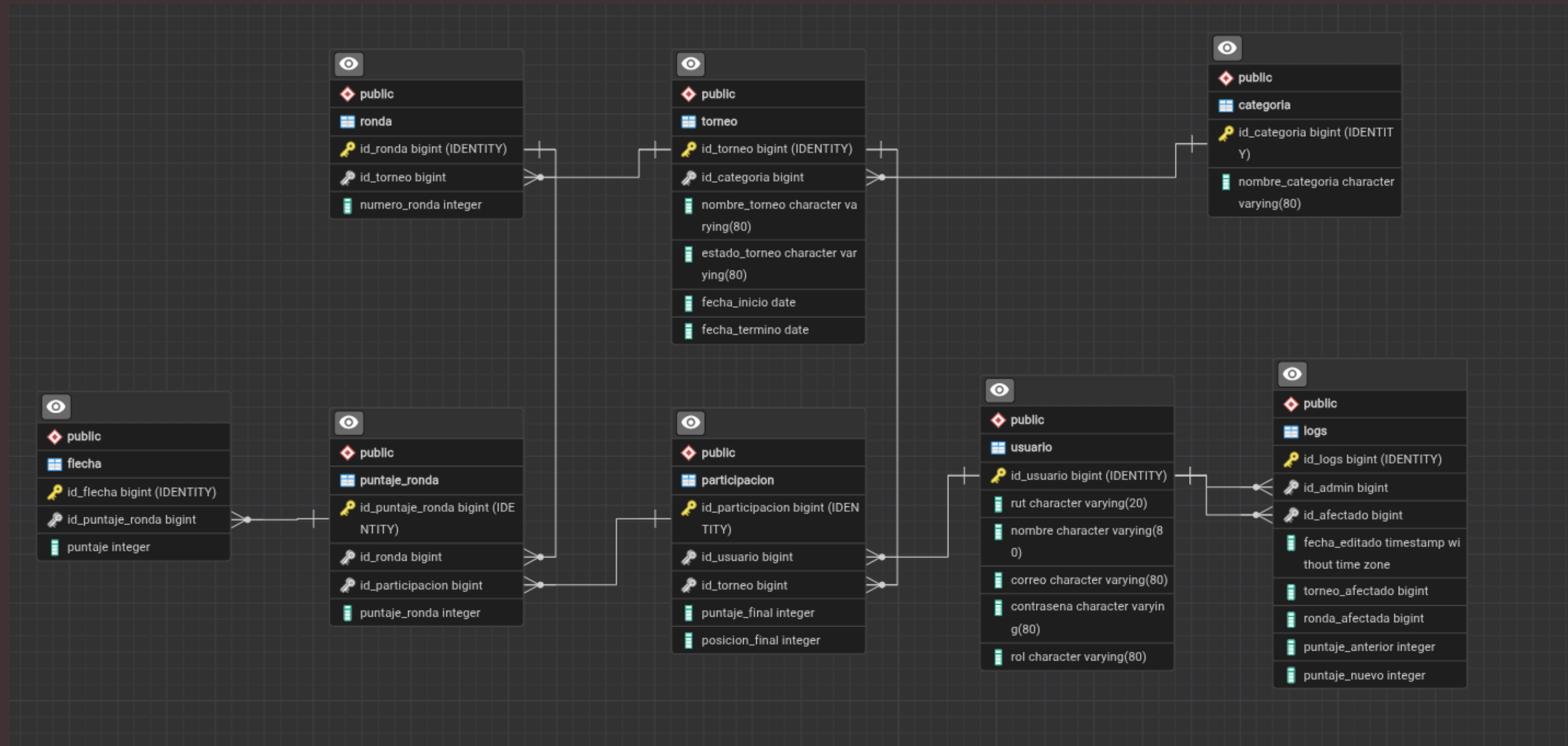
Frontend

Interfaz web intuitiva para arqueros y organizadores.

Demostración de funcionalidades



Modelo entidad relación



TRIGGER 1:

Impedir que se ingrese una puntuación de flecha superior a 10 o inferior a 0 (o "M").

```
-----TRIGGER 1-----  
  
CREATE OR REPLACE FUNCTION limites_puntaje_flecha_fn()  
RETURNS TRIGGER AS $$  
BEGIN  
  
    IF NEW.puntaje < 0 OR NEW.puntaje > 10 THEN  
        RAISE EXCEPTION 'Puntaje fuera de rango. Debe ser entre 0 y 10.';  
    END IF;  
  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER limites_puntaje_flecha  
BEFORE INSERT OR UPDATE ON flecha  
FOR EACH ROW  
EXECUTE FUNCTION limites_puntaje_flecha_fn();
```



TRIGGER 2:



```
-----TRIGGER 2-----
CREATE OR REPLACE FUNCTION audit_puntaje_fn()
RETURNS TRIGGER AS $$
DECLARE
    admin_id BIGINT;
    ronda_id BIGINT;
BEGIN
    IF OLD.puntaje_final IS DISTINCT FROM NEW.puntaje_final THEN

        admin_id := NULLIF(current_setting('my.user_id', true), '')::BIGINT;
        ronda_id := NULLIF(current_setting('my.ronda_id', true), '')::BIGINT;

        IF admin_id IS NULL THEN
            RAISE EXCEPTION 'No se configuró my.user_id para auditoría.';
        END IF;

        IF ronda_id IS NULL THEN
            RAISE EXCEPTION 'No se configuró my.ronda_id para auditoría.';
        END IF;

        INSERT INTO logs (
            id_admin,
            id_afectado,
            torneo_afectado,
            ronda_afectada,
            puntaje_anterior,
            puntaje_nuevo
        )
        VALUES (
            admin_id,
            OLD.id_usuario,
            OLD.id_torneo,
            ronda_id,
            OLD.puntaje_final,
            NEW.puntaje_final
        );

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER audit_puntaje
AFTER UPDATE OF puntaje_final ON participacion
FOR EACH ROW
EXECUTE FUNCTION audit_puntaje_fn();
```

Registrar en una tabla de auditoría
cada vez que un administrador
modifica la puntuación final de un
arquero.

Procedimiento Almacenado 1

```
--Procedimiento almacenado 1
CREATE OR REPLACE PROCEDURE registrar_puntaje_ronda(
    p_id_ronda BIGINT,
    p_id_participacion BIGINT,
    p_puntajes_flechas DECIMAL[],
    p_id_admin BIGINT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_torneo_id BIGINT;
    v_estado VARCHAR(80);
    v_puntaje_total DECIMAL := 0;
    v_id_puntaje_ronda BIGINT;
    v_puntaje DECIMAL;
BEGIN

    -- contexto para trigger
    PERFORM set_config('my.user_id', p_id_admin::text, true);
    PERFORM set_config('my.ronda_id', p_id_ronda::text, true);

    -- 1. Obtener torneo
    SELECT id_torneo INTO v_torneo_id
    FROM ronda
    WHERE id_ronda = p_id_ronda;

    -- 2. Validar estado
    SELECT estado_torneo INTO v_estado
    FROM torneo
    WHERE id_torneo = v_torneo_id;

    IF v_estado <> 'IN_COURSE' THEN
        RAISE EXCEPTION 'El torneo no está en curso';
    END IF;
```

```
-- 3. Calcular total
FOREACH v_puntaje IN ARRAY p_puntajes_flechas LOOP
    IF v_puntaje < 0 OR v_puntaje > 10 THEN
        RAISE EXCEPTION 'Puntaje inválido: %', v_puntaje;
    END IF;
    v_puntaje_total := v_puntaje_total + v_puntaje;
END LOOP;

-- 4. UPSERT puntaje_ronda
INSERT INTO puntaje_ronda (id_ronda, id_participacion, puntaje_ronda)
VALUES (p_id_ronda, p_id_participacion, v_puntaje_total)
ON CONFLICT (id_ronda, id_participacion)
DO UPDATE SET puntaje_ronda = EXCLUDED.puntaje_ronda
RETURNING id_puntaje_ronda INTO v_id_puntaje_ronda;

-- 5. Limpiar flechas anteriores (si existían)
DELETE FROM flecha
WHERE id_puntaje_ronda = v_id_puntaje_ronda;

-- 6. Insertar flechas nuevas
FOREACH v_puntaje IN ARRAY p_puntajes_flechas LOOP
    INSERT INTO flecha (id_puntaje_ronda, puntaje)
    VALUES (v_id_puntaje_ronda, v_puntaje);
END LOOP;

-- 7. Recalcular puntaje_final (CLAVE)
UPDATE participacion p
SET puntaje_final = (
    SELECT COALESCE(SUM(pr.puntaje_ronda), 0)
    FROM puntaje_ronda pr
    WHERE pr.id_participacion = p.id_participacion
)
WHERE p.id_participacion = p_id_participacion;

END;
$$;
```

Registrar la puntuación de una ronda completa (múltiples flechas) validando que el torneo esté activo. Page 08

Procedimiento Almacenado 2



```
-- Procedimiento almacenado 2
-- Procedimiento almacenado para actualizar las posiciones de un torneo después de que
-- se hayan ingresado todos los puntajes de las rondas.
CREATE OR REPLACE PROCEDURE actualizar_posiciones(p_id_torneo BIGINT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_torneo torneo.estado_torneo%TYPE;
BEGIN

    -- verificar torneo
    SELECT t.estado_torneo
    INTO v_estado_torneo
    FROM torneo t
    WHERE t.id_torneo = p_id_torneo;

    IF v_estado_torneo IS NULL THEN
        RAISE EXCEPTION 'El torneo no existe.';
    END IF;

    IF v_estado_torneo <> 'COMPLETED' THEN
        RAISE EXCEPTION 'El torneo no ha finalizado.';
    END IF;

    -- actualizar ranking
    UPDATE participacion p
    SET posicion_final = ranking.posicion
    FROM (
        SELECT
            id_participacion,
            DENSE_RANK() OVER (
                ORDER BY COALESCE(puntaje_final, 0) DESC,
                id_participacion ASC
            ) AS posicion
        FROM participacion
        WHERE id_torneo = p_id_torneo
    ) ranking
    WHERE p.id_participacion = ranking.id_participacion;
END;
$$;
```

Vista Materializada Leaderboard Histórico

```
-- Leaderboard Historico -> Promedio de puntos por flecha de Los 50 arqueros -> Actualizacion diaria
CREATE MATERIALIZED VIEW leaderboard_top_50 AS
SELECT
    DENSE_RANK() OVER (
        | ORDER BY promedio_puntos_flecha DESC
    ) AS posicion,
    id_usuario,
    nombre,
    promedio_puntos_flecha
FROM (
    SELECT u.id_usuario, u.nombre, COALESCE(AVG(f.puntaje), 0) AS promedio_puntos_flecha
    FROM usuario u
    JOIN participacion p ON p.id_usuario = u.id_usuario
    JOIN puntaje_ronda pr ON pr.id_participacion = p.id_participacion
    JOIN flecha f ON f.id_puntaje_ronda = pr.id_puntaje_ronda
    WHERE p.posicion_final IS NOT NULL
    GROUP BY u.id_usuario, u.nombre
) ranking
ORDER BY promedio_puntos_flecha DESC
LIMIT 50;

CREATE UNIQUE INDEX idx_leaderboard_top_50_usuario
ON leaderboard_top_50(id_usuario);
```

Rendimiento Mensual (Req. 9)

Basado en torneos finalizados los últimos 30 días

Pos	Arquero	Promedio Puntos
1°	Arquero 34	9.59
2°	Arquero 52	9.48
3°	Arquero 4	9.41
4°	Arquero 64	9.39
5°	Arquero 16	9.39
6°	Arquero 40	9.33
7°	Arquero 46	9.31
8°	Arquero 70	9.31
9°	Arquero 58	9.30
10°	Arquero 22	9.26
11°	Arquero 10	9.16
12°	Arquero 28	9.13

Endpoint para listar los arqueros con mejor rendimiento en el último mes.

```
// requerimiento 9: mejores del mes
public List<LeaderboardDTO> obtenerMejoresArquerosUltimoMes() { 1 usage Joseardi
    String sql = """
        SELECT
            u.id_usuario,
            u.nombre,
            COALESCE(AVG(f.puntaje), 0) AS promedio_puntos_flecha,
            DENSE_RANK() OVER (ORDER BY AVG(f.puntaje) DESC) as posicion
        FROM usuario u
        JOIN participacion p ON p.id_usuario = u.id_usuario
        JOIN torneo t ON p.id_torneo = t.id_torneo
        JOIN puntaje_ronda pr ON pr.id_participacion = p.id_participacion
        JOIN flecha f ON f.id_puntaje_ronda = pr.id_puntaje_ronda
        WHERE t.estado_torneo = 'COMPLETED'
            AND t.fecha_termino >= CURRENT_DATE - INTERVAL '1 month'
        GROUP BY u.id_usuario, u.nombre
        ORDER BY promedio_puntos_flecha DESC
    """;

    return jdbcTemplate.query(sql, ( ResultSet rs, int rowNum) ->
        new LeaderboardDTO(
            rs.getLong(columnLabel: "id_usuario"),
            rs.getString(columnLabel: "nombre"),
            rs.getDouble(columnLabel: "promedio_puntos_flecha"),
            rs.getInt(columnLabel: "posicion")
        )
    );
}
```

Copa Nacional 123

COMPLETED

Volver

Inscritos (8)

1 Arquero 1010000010-K

2 Arquero 2610000026-K

3 Arquero 810000008-K

4 Arquero 4510000045-K

5 Arquero 1510000015-K

6 Arquero 5510000055-K

Gestión de Flechas

Torneo Finalizado

Los puntajes han sido registrados y el ranking calculado.

Podio de Ganadores



Arquero 10

Lugar 1

10000010-K



Arquero 15

Lugar 2

10000015-K



Arquero 8

Lugar 3

10000008-K

Endpoint que reciba el ID de un torneo y devuelva el podio (1er, 2do y 3er lugar) usando el cálculo de la base de datos.

```
public List<InscritoDTO> obtenerPodio(Long idTorneo) {  
    String sql = ""  
        SELECT p.id_participacion, u.id_usuario, u.rut, u.nombre  
        FROM participacion p  
        JOIN usuario u ON p.id_usuario = u.id_usuario  
        WHERE p.id_torneo = ? AND p.posicion_final <= 3  
        ORDER BY p.posicion_final ASC  
        "";  
  
    return jdbcTemplate.query(sql, (ResultSet rs, int rowNum) -> {  
        InscritoDTO dto = new InscritoDTO();  
        dto.setIdParticipacion(rs.getLong(columnLabel: "id_participacion"));  
        dto.setIdUsuario(rs.getLong(columnLabel: "id_usuario"));  
        dto.setRut(rs.getString(columnLabel: "rut"));  
        dto.setNombre(rs.getString(columnLabel: "nombre"));  
        return dto;  
    }, idTorneo);  
}
```

Page 12

La Lógica del Negocio: El Backend

El Backend es la "**mente**" del sistema: procesa solicitudes, aplica reglas de negocio y coordina la comunicación entre el Frontend y la Base de Datos.



Gestión de Usuarios

Registro y autenticación con JWT para arqueros y organizadores.



CRUD Completo

Operaciones sobre todas las tablas del modelo, con paginación y queries simples.



Cálculo de Clasificaciones

Procesamiento automático de puntuaciones y generación de tablas de resultados.



API REST

Servicio REST estructurado para la comunicación eficiente con el Frontend.

¿Cómo se Conectan? La Comunicación Cliente-Servidor



Cada acción del usuario —como inscribirse a un torneo o consultar resultados— desencadena una cadena de comunicaciones entre las tres capas del sistema, garantizando integridad y consistencia de los datos.

Arquitectura del Sistema

Tres capas perfectamente integradas

El sistema opera como una unidad cohesionada donde cada capa tiene responsabilidades definidas:

- **Frontend:** Presentación e interacción con el usuario
- **Backend:** Reglas de negocio y servicio REST
- **Base de Datos:** Persistencia, integridad y consultas avanzadas

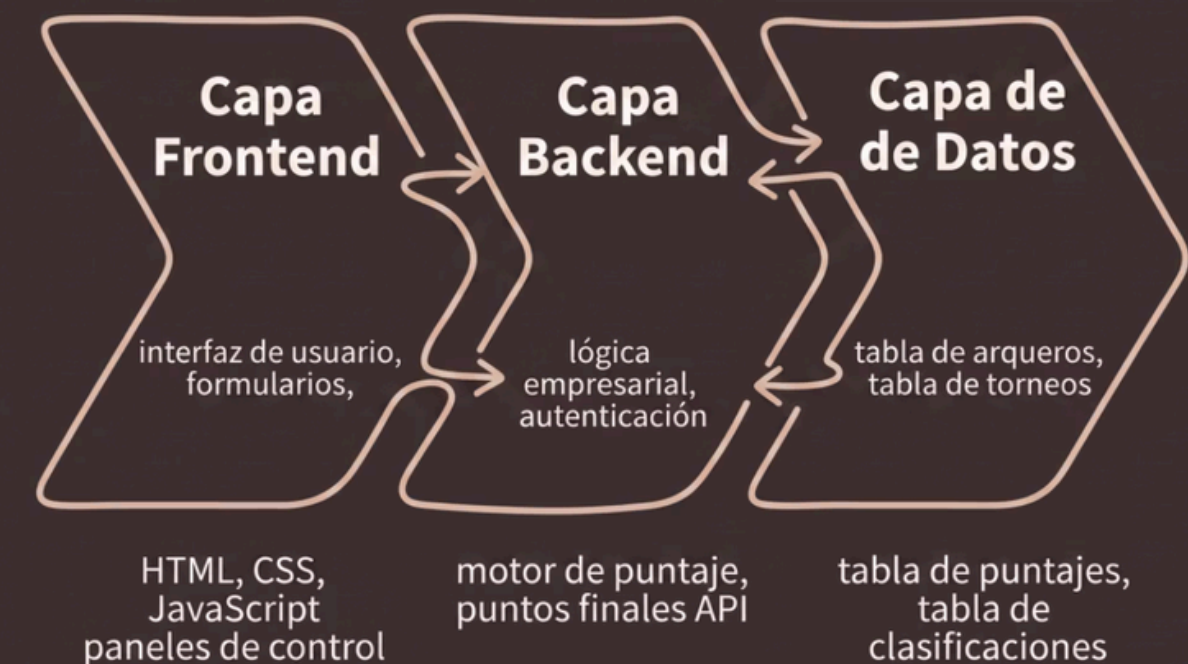
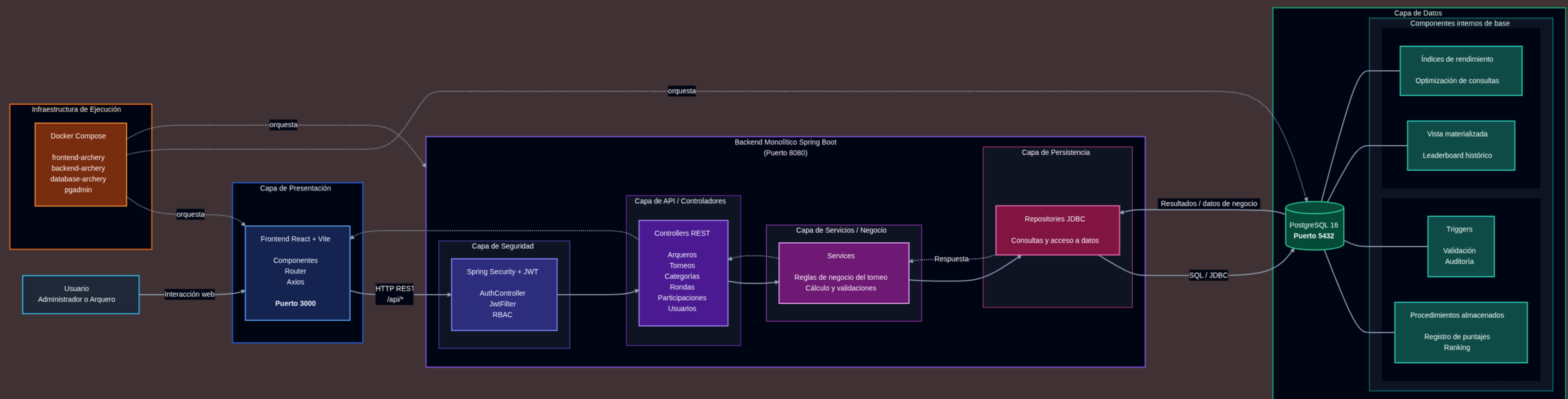


Diagrama arquitectónico del sistema



El Valor para Nuestros Usuarios

Para los Organizadores

- Administración centralizada y simplificada
- Eliminación de errores manuales
- Acceso a historial completo de competencias
- Reportes automáticos de resultados
- Logs para auditoria

Para los Arqueros

- Inscripción rápida y sin complicaciones
- Seguimiento transparente de resultados
- Perfil personal con historial de torneos
- Clasificaciones actualizadas en tiempo real



La Cara Visible: El Frontend

Una interfaz web intuitiva desarrollada con **HTML**, **CSS**, **JavaScript** y frameworks modernos, diseñada para ofrecer una experiencia fluida tanto a arqueros como a organizadores.

Registro e Inscripción

Formularios claros para crear cuenta y participar en torneos.

Calendario de Torneos

Visualización de eventos próximos y pasados.

Puntuaciones en Tiempo Real

Seguimiento en vivo de resultados y clasificaciones.

Perfiles de Arqueros

Historial personal de competiciones y rendimiento.



CONCLUSIONES

Problemas

- Iteraciones desarrollo agil
- Muchos cambios arquitecturales
- Coordinación inicial difusa
- División de tareas
- Desconocimiento de JDBC

Soluciones / aprendizajes

- Distribución 2-2-2
- Coordinación constante
- Aviso ante COMMIT
- Esqueleto de la aplicación antes.
- Manejo sin ORMs (JPA)

Resultados

- Buen flujo de trabajo con Github
- Integración Exitosa Sin ORM
- Seguridad y Escalabilidad JWT